

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

|                  |                                                                                                                |               |                                               |
|------------------|----------------------------------------------------------------------------------------------------------------|---------------|-----------------------------------------------|
| Course Code:     | CSA06                                                                                                          | Course Title: | Design and Analysis of Algorithms             |
| CO Assessed:     | CO2 – Manipulation (BL1, BL2, BL3)                                                                             | Assessment:   | Assessment Tool 2 – Scenario Based Assessment |
| Weightage:       | 20% of CIA                                                                                                     | Total Marks:  | 100                                           |
| CO2 Statement:   | Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3) |               |                                               |
| Student Name:    | M VishnuVardhan                                                                                                | Reg. No.:     | 192472087                                     |
| Section / Batch: | 2024                                                                                                           | Date:         | 28/07/2026                                    |

### Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

| Question Type             | Focus Area                                                                                                                   | Questions |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| Scenario Based Assessment | Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems | Q1        |

### SECTION A – SCENARIO BASED ASSESSMENT

**Q45. A plagiarism detection tool compares patterns across multiple large documents using brute-force string matching.**

- Explain how the matching process works.**
- Analyze the time complexity.**
- Discuss inefficiency in large-scale comparisons.**

### Code of Conduct

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

## RUBRICS FOR CALCULATION

| Criteria                                 | Wt. | Level 4 – Exemplary                                                                  | Level 3 – Proficient                                  | Level 2 – Developing                                      | Level 1 – Beginning                                     |
|------------------------------------------|-----|--------------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------|
| Problem Understanding & Context Analysis | 25% | Thoroughly understands the scenario with accurate interpretation of all requirements | Clearly understands the scenario with minor gaps      | Basic understanding; some aspects of the scenario unclear | Poor understanding or misinterpretation of the scenario |
| Application of Concepts                  | 30% | Applies appropriate concepts effectively to solve complex real-world problems        | Applies relevant concepts correctly with minor errors | Applies basic concepts but with limited accuracy          | Fails to apply appropriate concepts                     |
| Analytical & Decision-Making Skills      | 20% | Demonstrates strong analysis and selects optimal solutions with justification        | Good analysis with reasonable solutions               | Limited analysis; solutions lack justification            | Poor or no analysis; incorrect decisions                |
| Solution Design & Approach               | 15% | Well-structured, innovative, and feasible solution approach                          | Logical and structured solution with minor gaps       | Basic solution with limited structure or feasibility      | Unclear or incorrect solution approach                  |
| Clarity, Justification & Presentation    | 10% | Clear, well-justified explanation with strong reasoning                              | Clear explanation with some justification             | Limited clarity and weak justification                    | Poorly explained with no justification                  |

## Marks Evaluation:

| Criteria                                 | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|------------------------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Problem Understanding & Context Analysis | 25% |                     |                      |                      |                     |
| Application of Concepts                  | 30% |                     |                      |                      |                     |
| Analytical & Decision-Making Skills      | 20% |                     |                      |                      |                     |
| Solution Design & Approach               | 15% |                     |                      |                      |                     |
| Clarity, Justification & Presentation    | 10% |                     |                      |                      |                     |
| <b>Total Marks (100):</b>                |     |                     |                      |                      |                     |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name:             | Name:              | Name:              |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

## Q45. Plagiarism Detection Using Brute-Force String Matching

### a) How the Matching Process Works

Brute-force string matching (also known as the Naive String Matching algorithm) works by comparing a **pattern** (a suspected copied phrase or sentence, of length  $m$ ) against a **text** (the document being checked, of length  $n$ ), checking every possible position where the pattern could align with the text.

#### Step-by-step process:

1. Place the pattern at the very first position (index 0) of the text.
2. Compare the pattern with the text character-by-character at that position.
3. If **all** characters match, a match is recorded — this indicates a potentially plagiarized segment.
4. If a **mismatch** occurs at any character, stop comparing at that position and shift the pattern **one position to the right**.
5. Repeat the character-by-character comparison at the new position.
6. Continue this process until the pattern has been tested against every valid position in the text — that is, from index 0 up to index  $(n - m)$ .

#### Example:

Text: "the quick brown fox jumps", Pattern: "brown"

- Position 0: t vs b → mismatch → shift right by 1
- Position 1: h vs b → mismatch → shift right by 1
- ... continue shifting one character at a time ...
- Position 10: b-r-o-w-n vs b-r-o-w-n → **full match found**

**In the context of plagiarism detection**, this process is applied at a much larger scale:

- Every sentence or phrase (pattern) extracted from one document is compared against every position in every other document (text) in the collection.
- This is repeated across **all pairs of documents**, since any document could potentially be copied from, or copied into, any other document in the dataset.

### b) Time Complexity Analysis

**For a single pattern (length  $m$ ) against a single text (length  $n$ ):**

| Case         | Description                                                                                                           | Complexity                  |
|--------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------|
| Best Case    | Mismatch occurs at the first character for most alignments                                                            | $O(n)$                      |
| Worst Case   | Highly repetitive characters cause near-complete comparisons at every shift (e.g., pattern "aaaa" in text "aaaaaaaa") | $O(m \times n)$             |
| Average Case | Typical natural language text with occasional partial matches                                                         | $O(n)$ , bounded by $O(mn)$ |

**For full-scale plagiarism detection**, we must also account for the number of documents:

- Let  $k$  = number of documents, each of average length  $n$ .
- Comparing every document against every other document requires checking  **$k^2$  pairs**.
- Total complexity becomes:  **$O(k^2 \times m \times n)$**

This shows that the overall cost grows **quadratically with the number of documents** and **linearly-to-quadratically with document length**, making brute-force matching very expensive as the corpus grows.

### c) Inefficiency in Large-Scale Comparisons

Brute-force string matching, while conceptually simple, becomes highly inefficient for real-world plagiarism detection systems due to several factors:

1. **Quadratic growth with document count:**

Checking all pairs of documents ( $k^2$  comparisons) means that doubling the number of documents in the database roughly **quadruples** the total computational work — this does not scale for large repositories like academic databases with millions of submissions.

2. **Redundant re-scanning of characters:**

When a mismatch occurs, the naive algorithm shifts the pattern by only one position and **starts comparing from scratch**, discarding all information gained from the previous (partial) comparison. This wastes computation, especially in text with repeated substrings.

3. **No use of preprocessing or data structures:**

Brute-force matching does not build any auxiliary structures (like hash tables, suffix trees, or precomputed prefix tables) to speed up repeated searches — every new comparison starts completely fresh.

4. **Poor scalability for real-world data volumes:**

Academic and web-scale plagiarism checkers must compare documents containing thousands of words against databases with millions of documents. At  $O(k^2 \times m \times n)$  complexity, this becomes **computationally infeasible** within reasonable time limits.

5. **Better alternatives used in practice:**

Real plagiarism detection tools avoid brute-force matching by using:

- **Rabin-Karp algorithm** (hashing-based matching for faster substring comparison),
- **Suffix trees/arrays** (for efficient repeated substring searches),
- **Fingerprinting techniques like Winnowing** (selecting representative hashes of text chunks to compare documents quickly without full character-by-character checks).

These approaches reduce the effective search complexity closer to **linear or near-linear time**, making large-scale plagiarism detection practically feasible.